



Università del Piemonte Orientale

Dipartimento di Scienze e Innovazione  
Tecnologica

**Corso di Laurea in Informatica**

Tesi di Laurea

**TECNICHE DI DEEP LEARNING PER  
L'ANALISI STRUTTURALE DEL  
CLINKER**

**Relatore:**

Prof. Luca Piovesan

**Candidato:**

Simone Nardella

**Anno Accademico 2025/2026**

# Abstract

La produzione del cemento rappresenta una delle attività industriali più energivore e inquinanti a livello globale, rendendo cruciale l'ottimizzazione dei processi per ridurre l'impronta ecologica.

Attualmente, i test di controllo qualità sul clinker (il componente base del cemento) richiedono diverse ore. Questo ritardo obbliga gli operatori a mantenere i forni a temperature di sicurezza inutilmente elevate, causando un forte spreco di energia e producendo un materiale più denso e difficile da macinare. Per superare queste inefficienze, abbiamo sviluppato dei modelli di *Deep Learning* su immagini al microscopio di sezioni di clinker, con l'obiettivo di ottimizzare i costi di produzione ed evitare gli sprechi. La nostra ricerca si è concentrata su due obiettivi principali: la classificazione dei parametri di cottura, al fine di ricavare istantaneamente le condizioni termiche del forno, e la segmentazione dei componenti interni al clinker, per individuare i materiali presenti e calibrare l'energia necessaria alla successiva fase di macinazione.

La sperimentazione ha mostrato ottime performance dei modelli nella task di classificazione dei parametri di cottura, mentre la segmentazione si è confermata una sfida più complessa, ottenendo risultati inferiori che evidenziano la necessità di ulteriore ricerca futura.

# Capitolo 1

## Introduzione

La produzione del cemento è una delle attività industriali più inquinanti ed energivore al mondo, posizionandosi come la terza fonte di emissioni di anidride carbonica di difficile abbattimento a livello globale[1]. Di conseguenza, diventa cruciale ottimizzare i processi esistenti supportando maggiormente gli operatori che si occupano di gestire questi impianti.

Nello specifico, l'obiettivo di questa tesi è sviluppare un sistema di supporto per due attività chiave che riguardano l'analisi microscopica di un componente fondamentale del cemento: il clinker. Il clinker è un granulo roccioso che viene prodotto cuocendo calcare e argilla in forni industriali ad alte temperature e successivamente macinato.

Attualmente, i test di controllo qualità dei clinker tradizionali richiedono dalle 2 alle 4 ore[2] per restituire i risultati. Questo ritardo porta gli operatori a non avere alcuna informazione sullo stato di qualità del clinker durante la produzione, obbligandoli a mantenere i forni a temperature di "sicurezza" molto più elevate del necessario, causando un forte spreco energetico e producendo un materiale più denso, che richiederà a sua volta maggiori quantità di energia per essere successivamente macinato da mulini rotativi.

Per abbattere le inefficienze di questo processo, la nostra idea è di introdurre modelli di *Deep Learning* per l'elaborazione in tempo reale delle immagini al microscopio elettronico di sezioni di clinker. In questo modo, fornendo risultati rapidamente, puntiamo a ottimizzare i costi di produzione evitando sprechi e riducendo l'impronta ecologica.

Le due attività di supporto che ci siamo prefissati di svolgere sono:

- **Classificazione dei parametri di cottura:** Creare dei modelli che individuino in tempo reale i parametri di cottura del clinker data la sua immagine al microscopio. In questo modo, il sistema deduce all'istante le condizioni termiche del forno, permettendo agli operatori di regolarne la temperatura senza rischi, eliminando le temperature "assicurative" ed evitando sprechi di energia.
- **Segmentazione dei componenti interni:** Creare un modello che supporti l'utente nell'individuare i materiali di cui è composto il clinker,

assegnando un colore ad ogni componente interna. Quest'analisi permette di valutare velocemente l'energia necessaria alla rottura del clinker e calibrare la potenza dei mulini nella fase di macinazione successiva.

I risultati ottenuti dalla sperimentazione si sono rivelati promettenti. Per quanto riguarda la classificazione, i modelli addestrati hanno dimostrato un'ottima accuratezza nell'individuazione dei parametri di cottura, con pochi errori circoscritti a valori simili. Al contrario, la task di segmentazione si è rivelato più complesso e ha ottenuto risultati inferiori. Sebbene il post-processing ci abbia permesso di regolarizzare le predizioni, i modelli faticano ancora a discriminare componenti simili (come la Resina con i Pori o l'Alite con la Belite), evidenziando un bias verso le classi maggioritarie.

# Capitolo 2

## Lavoro svolto

### 2.1 Organizzazione del Progetto

Il nostro progetto di tesi è stato strutturato in due macro-aree distinte, volte a esplorare approcci di Computer Vision applicati a immagini al microscopio dei Clinker:

- **Classificazione dei Parametri di cottura:** La prima sezione del lavoro si occupa dello sviluppo di un modello di classificazione basato sull'architettura *Vision Transformer* (ViT). L'obiettivo è creare un sistema in grado di riconoscere, partendo da un'immagine micrografica della sezione interna del Clinker, i parametri fisici di cottura (es. Temperatura, Tempo di rampa) del campione. Il sistema non è composto da un singolo classificatore monolitico, ma da un ViT specializzato per ogni parametro target. A questi abbiamo affiancato un ulteriore modello dedicato all'identificazione dell'ID dell'esperimento.
- **Segmentazione semantica:** La seconda sezione affronta il problema dell'identificazione delle diverse componenti mineralogiche interne al Clinker (Alite, Belite, Fase Fusa, Pori/Imperfezioni, Resina, Background e Calce libera). Il processo manuale corrente è, infatti, molto oneroso e prone a errori; perciò, l'obiettivo è sviluppare un sistema di supporto automatico che permetta di ridurre drasticamente i tempi di lavoro attraverso un'analisi preliminare dell'immagine, lasciando all'operatore il compito di validare e raffinare i risultati. Per questa task abbiamo optato per un approccio ibrido: gli embeddings generati da una *Convolutional Neural Network* (CNN) vengono sfruttati per allenare un classificatore finale basato su algoritmi di *Gradient Boosting*, con lo scopo di etichettare ogni singolo pixel dell'immagine.

Entrambe le linee di ricerca sono state condotte sia su immagini a colori (RGB) che in scala di grigi. Questa scelta è dovuta al metodo di preparazione dei campioni: dopo il taglio e la lucidatura, infatti, le sezioni di Clinker vengono trattate con un acido che, a seconda del tempo di esposizione, altera

notevolmente i colori dei diversi elementi interni al campione. Pertanto, poiché il colore non rappresenta una qualità intrinseca delle diverse componenti mineralogiche, abbiamo deciso di testare i modelli anche in scala di grigi. In questo modo evitiamo di introdurre bias e forziamo le reti ad apprendere basandosi su caratteristiche distintive come la forma.

## 2.2 Classificazione con Vision Transformer

Abbiamo implementato il sistema di classificazione adottando l'architettura *Vision Transformer* (ViT). L'impiego di quest'architettura, preferita alle tradizionali *Reti Neurali Convoluzionali* (CNN), è motivato dalla sua capacità di catturare il contesto globale dell'immagine senza introdurre i bias induttivi tipici delle CNN. Questa caratteristica porta spesso i ViT a eguagliare o superare lo stato dell'arte nei compiti di classificazione di immagini [3]. Nel nostro caso, l'obiettivo è classificare le immagini rispetto a uno specifico attributo di cottura, sfruttando un *backbone* pre-addestrato e una *classification head* personalizzata.

### 2.2.1 Implementazione

#### 2.2.1.1 Acquisizione e Strutturazione del Dataset

Nella fase preliminare del lavoro, ci siamo occupati della raccolta e normalizzazione dei dati forniti dal partner industriale Buzzi. Il dataset risultante è composto da 1400 immagini, corredate dai relativi attributi di cottura. Quest'analisi iniziale ci ha permesso di individuare fin da subito due sfide significative per l'apprendimento automatico: una **scarsità dei dati** di addestramento e un marcato **sbilanciamento tra le classi**. Per mitigare queste criticità, abbiamo adottato strategie di regolarizzazione e bilanciamento, approfondite nel dettaglio nelle sezioni 2.2.1.3 e 2.2.2.

| Numero Esperimento | Rampa (°C/min) | T target (°C) | t (min) | Raffreddamento | Tempo rampa (min) | Tempo totale di permanenza (min) |
|--------------------|----------------|---------------|---------|----------------|-------------------|----------------------------------|
| 1                  | 25             | 1400          | 30      | lento          | 18                | 48                               |
| 2                  | 25             | 1400          | 30      | veloce         | 18                | 48                               |
| 3                  | 25             | 1400          | 30      | lento          | 18                | 48                               |
| 4                  | 25             | 1400          | 30      | veloce         | 18                | 48                               |
| 5                  | 40             | 1500          | 45      | veloce         | 13.75             | 58.75                            |
| 6                  | 10             | 1300          | 15      | lento          | 35                | 50                               |
| 7                  | 25             | 1400          | 30      | veloce         | 18                | 48                               |
| 8                  | 25             | 1400          | 30      | lento          | 18                | 48                               |
| 9                  | 40             | 1300          | 15      | veloce         | 8.75              | 23.75                            |
| 10                 | 10             | 1500          | 45      | lento          | 55                | 100                              |
| 11                 | 40             | 1500          | 15      | lento          | 13.75             | 28.75                            |
| 12                 | 10             | 1300          | 45      | veloce         | 35                | 80                               |
| 13                 | 10             | 1500          | 15      | veloce         | 55                | 70                               |
| 14                 | 40             | 1300          | 45      | lento          | 8.75              | 53.75                            |

Figura 2.1: Parametri esperimenti

### 2.2.1.2 Preparazione dei dati

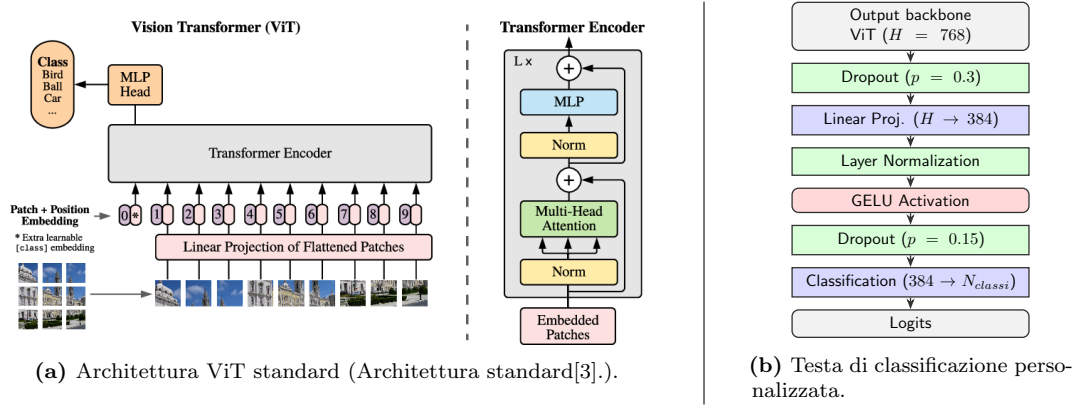
In questa fase, ci occupiamo innanzitutto di strutturare il dataset in base all'attributo target scelto per l'addestramento e successivamente di preprocessare le immagini grezze per convertirle in un formato più idoneo per il modello:

- **Mapping e Stratificazione:** Il sistema associa ogni immagine al relativo valore dell'attributo target su cui vogliamo addestrare il modello. Successivamente suddividiamo il dataset arricchito in set di addestramento e validazione tramite uno *split* stratificato (80/20) con seed fissato. In questo modo manteniamo invariata la distribuzione delle classi tra i due set, evitando l'esclusione delle classi più rare da uno dei due sottoinsiemi.
- **Preprocessing:** Le immagini grezze vengono elaborate attraverso i seguenti passaggi:
  1. *Cropping* dell'immagine, volto a eliminare le barre di scala presenti.
  2. Ridimensionamento alla dimensione nativa del ViT ( $224 \times 224$ ) e normalizzazione dei valori dei pixel su  $[-1, 1]$ .
  3. Conversione opzionale dell'immagine in scala di grigi (ottenuta replicando il canale di luminanza su 3 canali per mantenere la compatibilità con l'input RGB atteso dal modello).

### 2.2.1.3 Architettura Vision Transformer (ViT)

Per l'implementazione del modello abbiamo utilizzato il *backbone* "google/vit-base-patch16-224", sostituendo la testa di classificazione originale con una personalizzata:

- **Transfer learning:** Dato l'enorme numero di parametri del modello e la dimensione contenuta del dataset (1400 immagini), il processo di addestramento tenderebbe naturalmente all'overfitting. Per ovviare al problema abbiamo deciso di congelare i pesi dell'encoder ViT: in questo modo riduciamo drasticamente il numero di parametri addestrabili e riutilizziamo le conoscenze pregresse del modello.
- **Custom Classification Head:** La testa di classificazione originale (un singolo strato lineare) è stata sostituita da un *Multi-Layer Perceptron* (MLP) personalizzato a due strati. Questa architettura è stata scelta come compromesso tra due problematiche: un singolo strato sarebbe risultato troppo rigido e avrebbe limitato il modello a previsioni lineari, mentre una rete più profonda avrebbe introdotto un numero eccessivo di parametri, portando a un *overfitting*. La nostra *head* è quindi progettata per introdurre non linearità e massimizzare la regolarizzazione, e include: un primo strato di Dropout ( $p = 0.3$ ), una proiezione lineare che comprime le *feature* in uno spazio dimezzato ( $H/2$ ), una *Layer Normalization*, un'attivazione non lineare GELU e un Dropout finale ( $p = 0.15$ ) prima dello strato di output.



**Figura 2.2:** Architettura complessiva del modello. A sinistra (a), il modello ViT di base preso da [3] usato come estrattore di *feature* con pesi congelati. A destra (b), il dettaglio della testa di classificazione personalizzata in sostituzione a quella originale.

### 2.2.2 Strategia di training

Nella fase di addestramento di ciascun modello abbiamo seguito una strategia conservativa, volta a contrastare il bias naturale verso le classi maggioritarie. Al fine di trattare il problema dello sbilanciamento delle classi, introdotto nella sezione 2.2.1.1, non abbiamo optato per l'*oversampling*, che avrebbe introdotto ulteriore bias. Abbiamo scelto, invece, una Weighted Cross-Entropy Loss[4] (pesi inversi alla frequenza): in questa *loss function*, ciascuna classe del parametro target ha un peso inversamente proporzionale alla sua frequenza. Di conseguenza, il modello viene penalizzato maggiormente quando sbaglia la classificazione delle classi minoritarie, ed evita di predire sempre la classe di maggioranza. Tuttavia, poiché questa loss incentiva naturalmente il modello a produrre predizioni sempre con il massimo della confidenza (vicina al 100%), abbiamo applicato parallelamente un *Label Smoothing*[5] ( $\alpha = 0.1$ ) per mitigarne l'*overconfidence*. Infine, l'ottimizzazione dei pesi è affidata all'algoritmo **AdamW**[6] integrando un *Weight Decay* ( $10^{-4}$ ) e applicando un **Gradient Clipping**[7] (con norma max a 1.0) per prevenire l'instabilità numerica dei gradienti causata dai pesi elevati delle classi minoritarie. Per favorire la convergenza, abbiamo adottato un *Learning Rate Scheduler* con un *warmup* iniziale (100 step) e riduzione dinamica del tasso di apprendimento (fattore 0.5) in caso di plateau della loss. Durante l'addestramento il processo viene supervisionato da un meccanismo di *Early Stopping* (pazienza 15 epoche) valutato sul *validation set*. Come metrica abbiamo scelto esplicitamente l'*F1-Macro* al posto della classica *accuracy*. In contesti sbilanciati come il nostro, infatti, l'*accuracy* risulta ingannevole, poiché tende a premiare le prestazioni sulla classe maggioritaria. L'*F1-Macro* ci permette invece di valutare equamente le reali prestazioni del modello su tutte le classi dell'attributo target.



## 2.3 Segmentazione

Parallelamente alla classificazione dei parametri di cottura, il progetto affronta la sfida della segmentazione semantica. A differenza della classificazione globale, questa *task* opera a livello locale, assegnando a ciascun pixel dell'immagine una delle seguenti sei classi di interesse: Alite, Belite, Fase Fusa, Pori/Imperfezioni, Resina, Background e Calce libera.

### 2.3.1 Implementazione

Data l'enorme mole di dati da classificare (ogni singolo pixel delle immagini del dataset), per affrontare questa task, invece di addestrare una rete neurale *end-to-end* (computazionalmente onerosa e *data hungry*) abbiamo optato per un approccio ibrido che ci permettesse di mantenere alta la risoluzione nativa delle immagini con costi computazionali contenuti.

La nostra idea è stata di combinare la grande capacità delle reti convoluzionali nell'estrazione delle features con l'efficienza degli algoritmi di *Gradient Boosting* (descritti nella sezione 2.3.1.5) applicati su dati tabulari. La pipeline implementata si articola nelle seguenti tre fasi principali, che verranno analizzate nel dettaglio nelle sezioni successive:

1. **Backbone CNN e Decoder:** Inizialmente, utilizziamo una CNN pre-addestrata come *feature extractor* per estrarre le *feature maps* dalle immagini. Tuttavia, poiché i layer convoluzionali durante l'elaborazione applicano un *downsampling* spaziale, abbiamo aggiunto in coda al *backbone* un decoder che effettua l'*upsampling* delle mappe. Questo ci permette di fornire al classificatore successivo un vettore di features per ciascun pixel dell'immagine.
2. **Classificatore:** Le *feature maps* calcolate vengono date in input ad un classificatore basato su algoritmi di Gradient Boosting, che valuta i vettori di features assegnando una classe a ciascun pixel.
3. **Post-processing:** I risultati del classificatore vengono ulteriormente raffinati tramite un modello probabilistico *Conditional Random Field* (CRF) (in dettaglio nella sezione 2.3.1.6) per eliminare il rumore locale, uniformando le classificazioni nello spazio e delineandone meglio i bordi.

Considerato l'elevato costo computazionale richiesto da questa pipeline, per agevolare il testing e lo sviluppo abbiamo progettato il sistema per essere altamente parametrizzabile. Il codice permette infatti di configurare numerosi iperparametri tra cui: risoluzione feature map, numero di pixel considerati per immagine, numero di filtri del decoder, modello di backbone, algoritmo di *Gradient Boosting* e il numero di *trials* di ottimizzazione.

### 2.3.1.1 Gestione del Dataset e Preprocessing

Similmente alla task di classificazione, la prima parte del lavoro ha riguardato la raccolta e l'analisi del dataset fornito dal partner Buzzi. A conclusione di questo processo abbiamo ottenuto un dataset di 87 coppie immagine-maschera. Le criticità riscontrate sono simili a quelle della task precedente: un netto sbilanciamento tra classi ed una generale carenza di dati. In questo contesto, tuttavia, quest'ultima non si configura in un basso numero assoluto di pixel validi, ma piuttosto come una limitata variabilità dell'informazione, derivante tutta da sole 87 micrografie.

| Classe        | Nome              | Pixel Totali      | Percentuale    |
|---------------|-------------------|-------------------|----------------|
| 0             | Background        | 82,252,932        | 90.16%         |
| 1             | Resina            | 1,820,460         | 2.00%          |
| 2             | Pori/Imperfezioni | 1,034,328         | 1.13%          |
| 3             | Fase Fusa         | 883,692           | 0.97%          |
| 4             | Belite            | 1,657,733         | 1.82%          |
| 5             | Alite             | 3,094,398         | 3.39%          |
| 6             | Calce libera      | 482,569           | 0.53%          |
| <b>TOTALE</b> |                   | <b>91,226,112</b> | <b>100.00%</b> |

**Tabella 2.1:** Riepilogo finale della distribuzione dei pixel per ciascuna classe del dataset. Il background coincide con i pixel non etichettati dall'operatore e quindi esclusi dal training

### 2.3.1.2 Preparazione dei dati

A differenza della classificazione, la task di segmentazione richiede l'utilizzo di coppie di dati allineate: la micrografia in input e la relativa maschera di *Ground Truth*. All'interno delle maschere, il valore di ogni pixel codifica tramite un intero la classe di appartenenza (es. 1 = Resina, 5 = Alite). In questa fase ci siamo occupati di uniformare, standardizzare e ampliare l'intero dataset. Per quanto riguarda la gestione dello *split* dei dati e la conversione in scala di grigi, la logica applicata è analoga a quanto visto per la classificazione tramite ViT in 2.2.1.2.

- **Caricamento e Standardizzazione Dimensionale:** Le immagini vengono caricate normalizzando i valori dei pixel nell'intervallo  $[0, 1]$  e successivamente ridimensionate tutte ad una risoluzione target (es.  $1024 \times 1024$ ), utilizzando interpolazione lineare per le immagini e *nearest-neighbor* per le maschere. Quest'ultima è fondamentale per mantenere validi i valori interni alla maschera (valori interi).
- **Data Augmentation:** Per incrementare la variabilità del *dataset* e mitigare il rischio di *overfitting*, durante la fase di *training* applichiamo diverse trasformazioni casuali (*flip*, rotazioni, *shift*, *scale* e variazioni

delle luminosità e contrasto), aumentando i campioni utili. Ciò porta il modello ad imparare feature di carattere più generale sulle diverse classi.

- **Binding:** Il *binding* immagine-maschera viene effettuato tramite *filename*: per ogni micrografia *.png* viene cercata la maschera *.tif* con lo stesso *stem*.

### 2.3.1.3 Estrazione delle Feature (Backbone CNN e Decoder)

La prima sezione dell'architettura si occupa dell'elaborazione delle micrografie al fine di produrre *embedding* arricchiti da fornire al classificatore finale.

Il cuore di questo modulo è costituito da un *Encoder* convoluzionale (come *ResNet50* e *ConvNeXt-Tiny*) pre-addestrato e congelato. L'obiettivo, in questa fase, non è riaddestrare la rete (che causerebbe un forte *overfitting* date le dimensioni del dataset), bensì sfruttare la sua conoscenza pregressa per estrarre vettori di *features* morfologiche di carattere generale (come bordi o pattern locali) da fornire al classificatore successivamente.

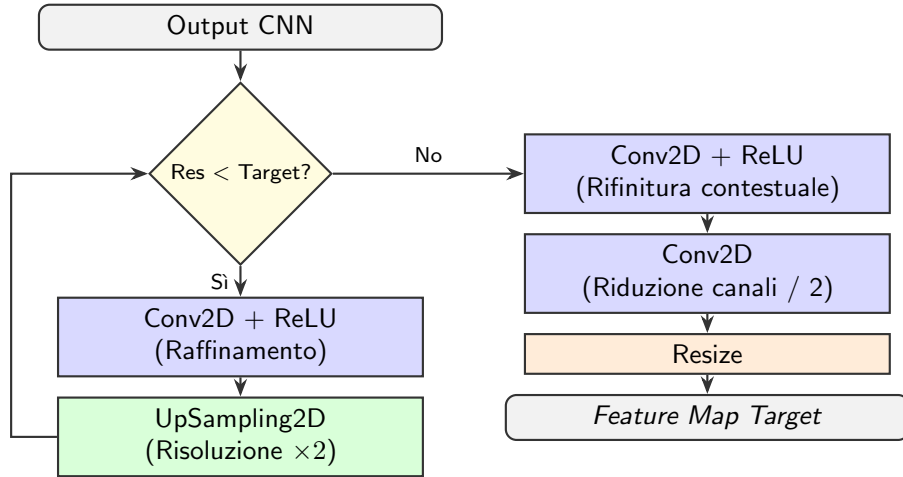
Tuttavia, una criticità intrinseca di tali reti è la drastica riduzione della risoluzione spaziale dovuta all'operazione di *pooling* durante l'elaborazione. In assenza di un meccanismo di ricostruzione, infatti, la segmentazione risulterebbe eccessivamente grossolana: ogni vettore di features in uscita codificherebbe un'intera regione (*patch*) dell'immagine, rendendo impossibile un allineamento preciso con le etichette a livello pixel presenti nelle maschere di *Ground Truth*.

Per ovviare a questo limite, abbiamo implementato un **Decoder dinamico** che proietta l'output a bassa risoluzione della CNN in uno spazio maggiore. Il decoder non ha un numero fisso di passaggi, ma raddoppia progressivamente la risoluzione della *feature map* fino al raggiungimento della dimensione desiderata. Ad ogni iterazione, il modulo applica in sequenza due *layer*:

1. **Uno strato convoluzionale (Conv2D):** con funzione d'attivazione RELU, il suo compito è quello di raffinare e ricombinare le features spaziali prima della loro espansione.
2. **Uno strato di *upsampling* (UpSampling2D):** che raddoppia le dimensioni della *feature map* mediante interpolazione.

Ottenuta una *feature map* con una dimensione finale pari o superiore a quella richiesta, la rete applica due strati Conv2D finali che si occupano di compiere un'ultima rifinitura contestuale e di dimezzare il numero di canali, rendendo così i vettori di *features* più compatti e computazionalmente gestibili per il classificatore. Infine applichiamo un *resize* finale che forza l'output ad assumere esattamente la risoluzione target nel caso il loop precedente l'avesse superata.

Come detto in precedenza, la struttura è parametrizzabile: la scelta del *backbone* (*ResNet50* o *ConvNeXt-Tiny*, il numero di canali del *decoder* e la risoluzione della *Feature Map* finale sono iperparametri configurabili, permettendo un controllo sul *trade-off* tra granularità della segmentazione e costo computazionale ricercato.



**Figura 2.3:** Architettura del *Decoder* dinamico. A sinistra il *loop* che raddoppia progressivamente la risoluzione della *Feature Map*. A destra, una volta raggiunta la dimensione richiesta, i *layer* per la rifinitura, compressione e *resize* finale.

#### 2.3.1.4 Sfide Computazionali e Campionamento (Pixel Sampling)

Durante la fase di addestramento del classificatore ci siamo imbattuti in un problema critico: l'enorme mole di dati. Una singola *feature map* generata dal modello può arrivare a contenere oltre un milione di patch (ad esempio, nel caso di una risoluzione di 1024x1024), ciascuna descritta da un vettore con una lunghezza corrispondente al numero di canali in uscita dal *Decoder* (es. 128 o 256, a seguito dell'operazione di dimezzamento finale). Applicando questo discorso sull'intero dataset, si ottiene un volume di dati intrattabile per l'elaborazione integrale in termini di memoria RAM.

Per rendere l'elaborazione computazionalmente sostenibile, abbiamo implementato un meccanismo di *Pixel Sampling*. Per ogni immagine processata, il sistema non considera l'intera matrice, ma estrae un sottoinsieme stratificato tra i pixel "validi" (escludendo background non etichettato). Il sottoinsieme di pixel considerati è un parametro liberamente configurabile dall'utente in base alle risorse hardware disponibili.

Questa tecnica non deve essere vista in opposizione alla Data Augmentation descritta nella sezione 2.3.1.2 ma anzi come un complemento. Infatti, mentre le trasformazioni geometriche arricchiscono la varianza delle *feature* estratte e irrobustiscono l'addestramento, il campionamento casuale ci assicura che il volume di dati rimanga computazionalmente sostenibile. Il risultato è un *training* più robusto ed efficiente.

#### 2.3.1.5 Classificazione Pixel

Una volta generata la *feature map*, i dati vengono convertiti in un formato tabulare: la mappa generata e la relativa maschera vengono "appiattite", ottenendo un dataset in cui ogni riga rappresenta una *patch* e le colonne le *features* calcolate dal modello.

Su questo nuovo dataset, addestriamo un classificatore basato su **Gradient Boosting** [8]. Il Gradient Boosting è una tecnica di apprendimento che costruisce un modello predittivo combinando in sequenza molteplici modelli deboli (tipicamente alberi decisionali con profondità limitata). L'esecuzione segue un flusso iterativo: ogni nuovo albero viene addestrato per minimizzare l'errore residuo commesso dagli alberi precedenti.

Questa scelta algoritmica è motivata dai risultati allo stato dell'arte ottenuti da questi modelli sui dati strutturati[9] come i nostri: i metodi di *Boosting* infatti eccellono nel catturare pattern complessi e garantiscono un'ottima generalizzazione, mitigando il rischio di *overfitting* che caratterizza le reti neurali quando addestrate su dataset limitati.

A livello implementativo, per garantire la massima flessibilità, la nostra pipeline supporta l'addestramento del classificatore sia utilizzando **XGBoost**[10] (generalmente associato a prestazioni predittive ottimali), sia **LightGBM**[11] (scelto per la sua efficienza e leggerezza computazionale).

### 2.3.1.6 Post-processing

Completato l'addestramento del modello ibrido, in fase di inferenza il classificatore restituisce una mappa contenente le probabilità di appartenenza di ogni *patch* alle diverse classi. Questa maschera viene successivamente riportata alla risoluzione originale dell'immagine tramite un'operazione di *upsampling* (con interpolazione lineare) e infine rinormalizzata.

Poiché il classificatore valuta le *feature* di ciascuna *patch* in modo indipendente l'una dall'altra, le maschere di segmentazione generate tendono a presentare bordi irregolari e il cosiddetto "*salt and pepper noise*", ovvero pixel isolati classificati in modo anomalo all'interno di regioni omogenee.

Per mitigare questo rumore, abbiamo integrato nella pipeline un ultimo passaggio di *post-processing* basato su **Fully Connected Conditional Random Field (DenseCRF)**.

Questo algoritmo modella l'immagine come un grafo, in cui ogni pixel rappresenta un nodo connesso con tutti gli altri. L'algoritmo raffina la segmentazione minimizzando il "costo" energetico globale della maschera, definito dal bilanciamento di due penalità contrastanti:

- **Potenziale Unario:** Rappresenta il costo di assegnare una specifica classe a un pixel, basandosi esclusivamente sulle probabilità fornite in output dal classificatore. Maggiore è la probabilità per una classe, minore sarà il costo della sua assegnazione.
- **Potenziale a Coppie:** Rappresenta una penalità aggiuntiva che scoraggia l'assegnazione di classi differenti a pixel vicini nello spazio e simili nel colore.

In termini pratici, l'applicazione del DenseCRF restituisce maschere più omogenee, riducendo il rumore e allineando i confini delle classi ai reali bordi dell'immagine.

Concluso questo processo, la mappa raffinata viene convertita nella maschera di segmentazione definitiva: a ciascun pixel viene assegnata la classe con la probabilità maggiore, generando così l'output finale del nostro modello.

### 2.3.2 Strategia di Training

Per il training, abbiamo scelto una strategia che bilanciassse e regolarizzasse l'apprendimento del classificatore per contrastare sia il forte *bias* dovuto allo sbilanciamento tra classi, sia il rischio di *overfitting* dovuto alla ridotta varietà del *dataset*. È importante specificare che tutti i pixel appartenenti al background (classe 0) sono stati esclusi dall'addestramento poiché rappresentano le aree delle maschere che non sono state annotate dall'operatore e su cui non abbiamo informazioni.

#### 2.3.2.1 Funzione Obiettivo e Metrica

Come loss abbiamo scelto la *Multi-class Logarithmic Loss* (`mlogloss`), con obiettivo basato su `multi:softprob`. Questa configurazione costringe il classificatore a produrre una distribuzione di probabilità per ogni classe, invece di restituire una singola etichetta. Questa scelta ci permette da un lato una convergenza più stabile durante l'addestramento, dall'altro fornisce tutte le informazioni necessarie al DenseCRF per calcolare il potenziale unario nella fase successiva di post processing.

#### 2.3.2.2 Gestione dello Sbilanciamento (Sample Weights)

Come anticipato in 2.3.1.1, le micrografie dei Clinker presentano un forte sbilanciamento naturale. Un addestramento standard porterebbe inevitabilmente il modello a ignorare le classi rare per minimizzare l'errore globale.

Per contrastare questo *bias*, abbiamo implementato un **Sample Weighting**. Il sistema calcola in anticipo un peso per ogni classe, inversamente proporzionale alla sua frequenza nelle maschere di *ground truth* del dataset.

Questo coefficiente viene poi fornito all'algoritmo di *gradient boosting*. Di conseguenza, durante la fase di *training*, la funzione di loss (`mlogloss`) penalizza in misura maggiore gli errori di classificazione commessi sulle classi rare, costringendo il classificatore a focalizzare l'apprendimento anche sui pattern delle classi minoritarie garantendo una classificazione più robusta.

#### 2.3.2.3 Regolarizzazione e Tuning degli Iperparametri

Data l'alta dimensionalità degli embeddings, il rischio di overfitting è elevato (a causa della cosiddetta "*Curse of dimensionality*"). Per garantire la robustezza del modello, abbiamo implementato un set aggressivo di iperparametri di regolarizzazione, tra cui: penalità L1 (`reg_alpha`) e L2 (`reg_lambda`) sui pesi delle foglie, vincoli sulla profondità massima degli alberi e sottocampionamento casuale delle features accessibili per ogni singolo albero (`colsample_bytree`).

La ricerca della configurazione ottimale (*tuning*) di questi iperparametri è stata automatizzata affidandosi al framework **Optuna** [12]. L'algoritmo esplora in modo mirato lo spazio di ricerca degli iperparametri minimizzando la mlogloss calcolata sul set di validazione. Infine, per evitare un addestramento ridondante, abbiamo integrato nel processo di *tuning* un *early stopping* con pazienza pari a 20 round.

# Capitolo 3

## Risultati

### 3.0.1 Risultati classificazione

Per il compito di classificazione, abbiamo valutato le performance dei modelli sui relativi attributi di cottura e comparato i risultati sia usando immagini a colori che in scala di grigi.

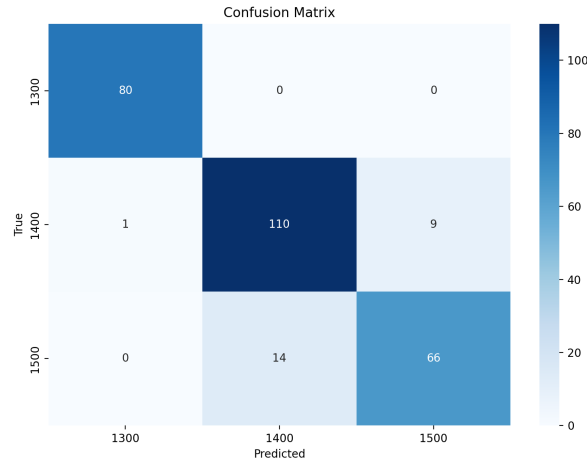
Nella Tabella 3.1 riportiamo le metriche di *Accuracy* e *Macro F1-Score* per ciascun attributo. Come si può vedere dai dati, l'utilizzo delle immagini in scala di grigi, non ha portato ad una grossa perdita nelle prestazioni, raggiungendo dei minimi solo per l'attributo "ID" e "Tempo Totale".

| Attributo                | A Colori |          | Scala di Grigi |          |
|--------------------------|----------|----------|----------------|----------|
|                          | Accuracy | Macro F1 | Accuracy       | Macro F1 |
| Attributo ID             | 0.786    | 0.784    | 0.650          | 0.643    |
| Attributo Raffreddamento | 0.829    | 0.828    | 0.718          | 0.718    |
| Attributo Rampa          | 0.886    | 0.884    | 0.811          | 0.804    |
| Attributo Temperatura    | 0.914    | 0.916    | 0.900          | 0.903    |
| Attributo Tempo          | 0.839    | 0.831    | 0.789          | 0.776    |
| Attributo Tempo Rampa    | 0.864    | 0.852    | 0.779          | 0.759    |
| Attributo Tempo Totale   | 0.800    | 0.774    | 0.732          | 0.686    |

**Tabella 3.1:** Confronto delle metriche di classificazione per i 7 attributi analizzati.

Per un'analisi più approfondita, prendiamo in considerazione i risultati sull'attributo "Temperatura" con immagini in scala di grigi come esempio. In Figura 3.1 possiamo osservare la matrice di confusione, che evidenzia come il modello sia in grado di riconoscere con ottima precisione le 3 classi di temperatura. Inoltre, gli errori commessi dal modello avvengono tra classi limitrofe.





**Figura 3.1:** Matrice di confusione per la classificazione dell'attributo Temperatura (modello Scala di grigi).

### 3.0.2 Risultati segmentazione

Per quanto riguarda la segmentazione, abbiamo valutato le performance usando diverse combinazioni possibili tra due backbone (ConvNext-Tiny e ResNet50) e due algoritmi di Gradient Boosting (XGBoost e LightGBM). Analogamente alla classificazione, riportiamo le metriche di Macro F1-Score, mostrando anche il miglioramento ottenuto tramite il post-processing con CRF (Conditional Random Field). Per garantire tempi di esecuzione gestibili con le risorse computazionali a disposizione, abbiamo impostato degli iperparametri conservativi. I risultati mostrati, quindi, sono da intendersi come un punto di partenza delle reali potenzialità del modello, e non come le sue performance definitive. Nello specifico, i modelli sono stati addestrati con i seguenti parametri:

- **Dimensione *feature map*:** 256x256
- **Pixel campionati:** 10.000
- **Dimensione batch:** 2
- **Filtri del decoder:** 128
- **Numero Trial Optuna:** 40

Nella Tabella 3.2 riportiamo i risultati ottenuti utilizzando le immagini a colori come input, mentre in 3.3 su immagini in scala di grigi.

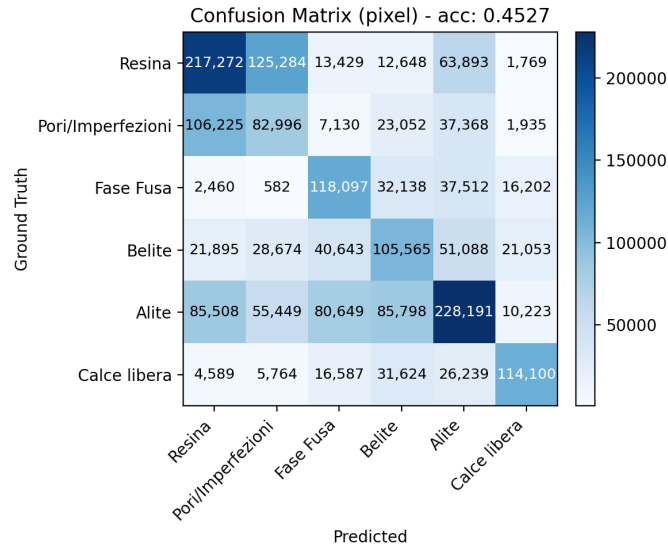
| Backbone      | Classificatore | Senza CRF | Con CRF |
|---------------|----------------|-----------|---------|
|               |                | mIoU      | mIoU    |
| ConvNeXt Tiny | XGBoost        | 0.289     | 0.303   |
| ConvNeXt Tiny | LightGBM       | 0.264     | 0.288   |
| ResNet50      | XGBoost        | 0.241     | 0.251   |
| ResNet50      | LightGBM       | 0.235     | 0.258   |

**Tabella 3.2:** Risultati della segmentazione su immagini a colori.

| Backbone      | Classificatore | Senza CRF | Con CRF |
|---------------|----------------|-----------|---------|
|               |                | mIoU      | mIoU    |
| ConvNeXt Tiny | XGBoost        | 0.253     | 0.266   |
| ConvNeXt Tiny | LightGBM       | 0.248     | 0.262   |
| ResNet50      | XGBoost        | 0.223     | 0.251   |
| ResNet50      | LightGBM       | 0.213     | 0.233   |

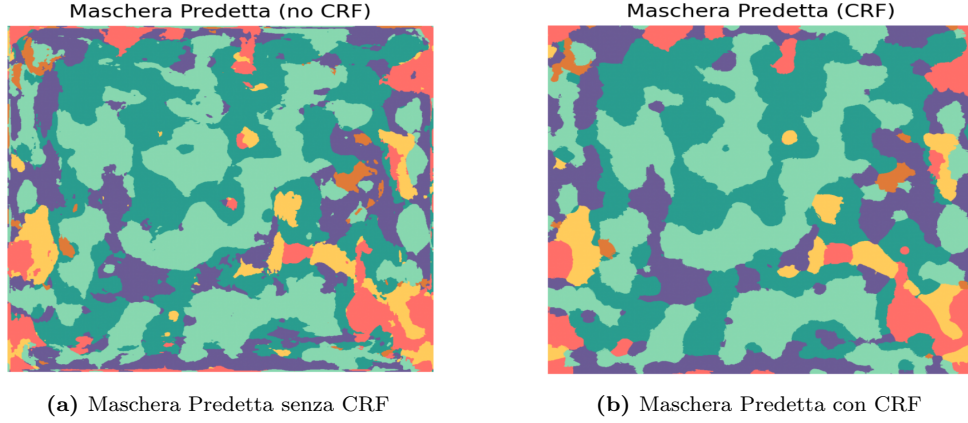
**Tabella 3.3:** Risultati della segmentazione su immagini in scala di grigi

Prendiamo ora in analisi, come esempio, la combinazione ConvNeXt-Tiny con XGBoost addestrata su immagini in scala di grigi. In Figura 3.2 è riportata la matrice di confusione. Possiamo notare come, sebbene le classi principali come Alite e Fase Fusa vengano individuate, il modello faticchi a discriminare classi visivamente molto simili tra loro, confondendo frequentemente la Resina con i Pori/Imperfezioni o mescolando Alite con Belite.



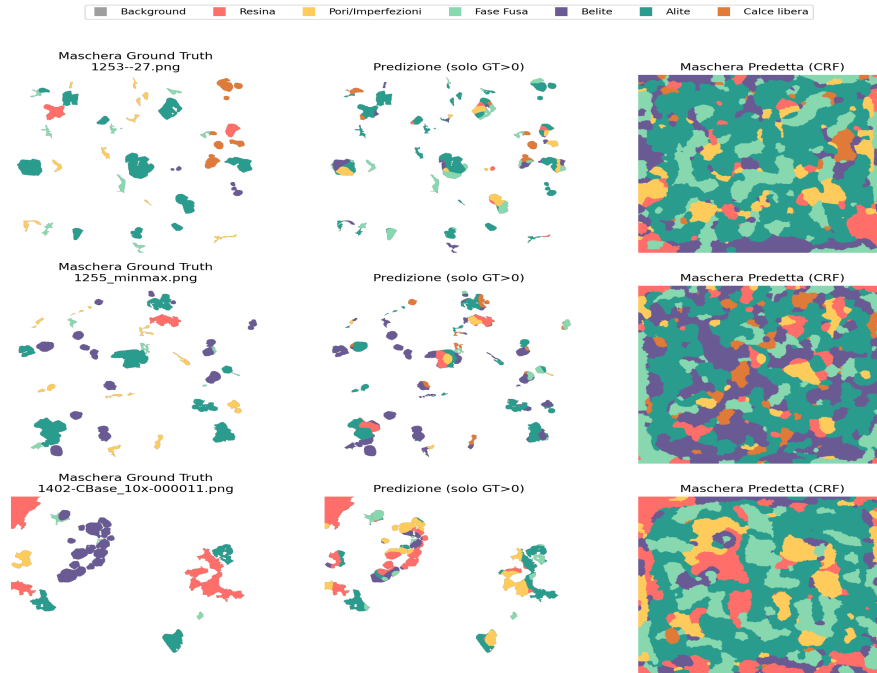
**Figura 3.2:** Matrice di confusione per la segmentazione (ConvNeXt-Tiny + XGBoost).

Osservando le predizioni in Figura 3.3, possiamo notare l'importanza del post-processing. Sebbene le predizioni grezze presentino del rumore, l'applicazione del CRF restituisce una maggiore coerenza nella previsione migliorando in linea generale l'inferenza.



**Figura 3.3:** Confronto tra le maschere predette con e senza l'uso di Conditional Random Fields (CRF).

Come visibile in Figura 3.4, il modello fatica a definire i bordi delle diverse componenti. Questo limite è attribuibile principalmente a due fattori: In primo luogo, l'uso di immagini in scala di grigi riduce indubbiamente il potenziale a coppie della CRF, che fatica a distinguere efficacemente i confini basandosi esclusivamente sulle variazioni di luminanza tra pixel adiacenti. In secondo luogo, influisce anche la scelta degli iperparametri di segmentazione che, come discusso in precedenza, non rappresentano la configurazione ottimale.



**Figura 3.4:** Esempi visivi di segmentazione. Da sinistra verso destra: Immagine originale, *Ground Truth*, Predizione sui soli pixel del *Ground Truth*, e Maschera finale.

# Capitolo 4

## Conclusioni

Come detto nel capitolo 1, l'ottimizzazione dei processi di produzione del cemento è una sfida di cruciale importanza nel panorama ecologico odierno. In questa tesi abbiamo affrontato tale problematica proponendo sistemi di supporto basati su modelli avanzati di *Deep Learning*, in particolare architetture *Vision Transformer* (ViT) e reti ibride, per automatizzare l'analisi microscopica del clinker.

L'esito della nostra sperimentazione mostra, da un lato, risultati promettenti nel campo della classificazione dei parametri di cottura, dall'altro lato la task di segmentazione si è rivelata decisamente più complessa, mostrando difficoltà nella delineazione dei confini e nel riconoscimento di classi visivamente simili, evidenziando il bisogno di ricercare ulteriormente.

È necessario, infine, specificare che le immagini utilizzate per l'addestramento, pur essendo numerose, sono state ricavate a partire da un numero ristretto di campioni fisici. Questa scarsa variabilità dei dati ha indubbiamente introdotto un *bias* nel dataset, inficiando, in parte, la capacità di generalizzazione dei modelli rispetto alle metriche ottenute.

In conclusione, il lavoro da noi svolto è solo un primo passo verso una soluzione ancora ampiamente sviluppabile. Per gli sviluppi futuri, sarà fondamentale ampliare il dataset con nuovi campioni di clinker che, insieme ad una maggiore disponibilità di risorse computazionali, permetterebbe di mitigare il bias e irrobustire decisamente di più i modelli. Un ulteriore passo futuro sarà sicuramente l'applicazione di questi sistemi in ambito industriale. La nostra ricerca, infatti, si è occupata di campioni prodotti in laboratorio con condizioni ambientali ben più controllate rispetto a quelle degli stabilimenti industriali. Si potrebbe quindi estendere questo lavoro per sperimentare le capacità dei modelli su campioni provenienti direttamente dagli stabilimenti, verificando se le performance rimangano ottimali o se sia necessario un *fine-tuning* ulteriore sui nuovi dati.

# Bibliografia

- [1] Steven J. Davis et al. «Net-zero emissions energy systems». In: *Science* 360.6396 (2018), eaas9793. DOI: 10.1126/science.aas9793. eprint: <https://www.science.org/doi/pdf/10.1126/science.aas9793>. URL: <https://www.science.org/doi/abs/10.1126/science.aas9793>.
- [2] Oxmaint. *AI-Driven Kiln Optimization in Cement Manufacturing*. <https://oxmaint.com/industries/manufacturing-plant/ai-driven-kiln-optimization-cement-manufacturing>. Accessed: 2026-04-12.
- [3] Alexey Dosovitskiy et al. «An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale». In: *International Conference on Learning Representations (ICLR)*. 2021.
- [4] Ian Goodfellow, Yoshua Bengio e Aaron Courville. *Deep Learning*. MIT Press, 2016.
- [5] Christian Szegedy et al. «Rethinking the inception architecture for computer vision». In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016, pp. 2818–2826.
- [6] Ilya Loshchilov e Frank Hutter. «Decoupled Weight Decay Regularization». In: *International Conference on Learning Representations (ICLR)*. 2019.
- [7] Razvan Pascanu, Tomas Mikolov e Yoshua Bengio. «On the difficulty of training recurrent neural networks». In: *Proceedings of the 30th International Conference on Machine Learning*. A cura di Sanjoy Dasgupta e David McAllester. Vol. 28. Proceedings of Machine Learning Research 3. Atlanta, Georgia, USA: PMLR, giu. 2013, pp. 1310–1318. URL: <https://proceedings.mlr.press/v28/pascanu13.html>.
- [8] Jerome H. Friedman. «Greedy Function Approximation: A Gradient Boosting Machine». In: *The Annals of Statistics* 29.5 (2001), pp. 1189–1232. ISSN: 00905364, 21688966. URL: <http://www.jstor.org/stable/2699986> (visitato il giorno 11/03/2026).
- [9] Leo Grinsztajn, Edouard Oyallon e Gael Varoquaux. «Why do tree-based models still outperform deep learning on typical tabular data?». In: *Advances in Neural Information Processing Systems*. A cura di S. Koyejo et al. Vol. 35. Curran Associates, Inc., 2022, pp. 507–520. URL: [https://proceedings.neurips.cc/paper\\_files/paper/2022/](https://proceedings.neurips.cc/paper_files/paper/2022/)

file/0378c7692da36807bdec87ab043cdadc - Paper - Datasets \_ and \_ Benchmarks.pdf.

- [10] Tianqi Chen e Carlos Guestrin. «Xgboost: A scalable tree boosting system». In: *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*. 2016, pp. 785–794.
- [11] Guolin Ke et al. «Lightgbm: A highly efficient gradient boosting decision tree». In: *Advances in neural information processing systems* 30 (2017).
- [12] Takuya Akiba et al. «Optuna: A next-generation hyperparameter optimization framework». In: *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*. 2019, pp. 2623–2631.

# Ringraziamenti

Voglio esprimere la mia profonda gratitudine al mio relatore, il Prof. **Luca Piovesan**, per la pazienza e il prezioso aiuto fornito durante l'intero sviluppo di questo progetto, senza il quale non sarebbe stato possibile realizzare questa tesi.

Un ringraziamento speciale va alla **mia famiglia**, per avermi sostenuto con amore in questo cammino, dandomi sempre nella quotidianità la serenità necessaria e il supporto nei momenti di crisi, non facendomi mai mancare nulla.

Ringrazio di cuore **Ingrid**, per avermi supportato (e sopportato) ogni giorno nonostante le mie paranoie in questo ultimo e tortuoso anno e mezzo. Grazie per essermi stata sempre vicina in ogni momento di bisogno, sempre con l'affetto e l'attenzione che ti contraddistinguono.

Ringrazio con affetto **Brian, Alessandro, Alice, Sober, Samu, Zucco, Sebastian, Amin, Riccardo e Miriam** per avermi fatto staccare nelle bellissime giornate in giro tra le montagne o su qualche ferrata tra le piccole pause dallo studio. Se ho ancora capelli in testa è merito vostro.

Un grazie va a **Marco, Arianna, Silvia e Vlad**, che hanno accompagnato le mie giornate in Università, dividendo con me lo studio e le *immancabili* pause caffè. Che bello che era venire in università sapendo di beccarvi sempre.

Ringrazio **Daniele** per la preziosissima compagnia in questi ultimi e molto pensosi mesi. Ne ho avuto davvero bisogno.

Un pensiero speciale va a **Linda, Matilda e Sophie**, per avermi addolcito le infinite giornate di studio a Novara in quello che è stato probabilmente uno dei periodi più difficili del mio intero percorso.

Infine ringrazio gli OG della **3k sulla street** per essere stati i miei compagni di disavventure fin dal primo giorno in cui ho iniziato questo lungo viaggio.

Grazie a tutti voi, in ogni vostra sfaccettatura. Non sarei mai arrivato a questo traguardo senza di voi, il merito è anche vostro.

Grazie davvero.